POSIX Threads

Definition

POSIX Threads, usually called **Pthreads**, is the IEEE standardized thread library for writing portable multithreaded programs. It is defined by IEEE standard `1003.1c` and is supported by most UNIX systems.

1. Why Pthreads Matter

Main Idea

Different operating systems may implement threads differently, so a standard API is needed to write portable thread programs. Pthreads provides that standard interface.

Scope

The full Pthreads standard defines many function calls. In practice, a smaller core set is enough to understand the basic thread model and how Pthreads programs are written.

2. Core Pthreads Calls

Main Calls

Some of the most important Pthreads calls are listed below.

Thread Call	Description
`pthread_create`	Create a new thread
`pthread_exit`	Terminate the calling thread
`pthread_join`	Wait for a specific thread to exit
`pthread_yield`	Release the CPU to let another thread run
`pthread_attr_init`	Create and initialize a thread attribute structure
`pthread_attr_destroy`	Remove a thread attribute structure

3. Properties of a Pthread

What Each Thread Has

Every Pthread has its own identity and execution context.

Typical Properties

A Pthread has:

- a thread identifier,
- a set of registers,
- a program counter,
- an attribute structure.

Thread Attributes

The attribute structure stores information such as:

- stack size,
- scheduling parameters,
- and other thread configuration details.

4. `pthread_create`

Purpose

`pthread_create` creates a new thread.

Key Idea

The newly created thread gets its own thread identifier, much like a new process gets a PID with `fork`. The thread identifier is later used in operations such as `pthread_join`.

Same Address Space

Unlike `fork`, the new thread does not get a new address space. It automatically runs inside the same process and shares the same memory with the creating thread.

5. `pthread_exit`

Purpose

When a thread finishes its assigned work, it can terminate itself by calling `pthread_exit`.

Effect

This stops the calling thread and releases its stack.

6. `pthread_join`

Purpose

Sometimes one thread must wait for another thread to finish. It does this using `pthread_join`.

Behavior

The calling thread blocks until the specific target thread terminates.

7. `pthread_yield`

Purpose

`pthread_yield` allows a thread to voluntarily give up the CPU so that another thread can run.

Why This Exists

Processes are usually viewed as competing for CPU time, but threads in the same process are usually cooperating. Because of that, sometimes the programmer wants one thread to deliberately let another thread run.

8. Attribute Functions

Initialization

`pthread_attr_init` creates a thread attribute structure and initializes it with default values.

Customization

After initialization, fields inside the attribute structure can be changed, for example to adjust priority or stack-related settings.

Cleanup

`pthread_attr_destroy` removes the attribute structure and frees the memory used for it. It does **not** destroy threads that are using those attributes.

9. Typical Thread Lifecycle

Stage	Pthreads Action
Prepare attributes	`pthread_attr_init`
Create thread	`pthread_create`
Run thread function	thread executes assigned work
Wait if needed	`pthread_join`
Terminate thread	`pthread_exit`
Destroy attributes	`pthread_attr_destroy`

10. Example Program

Main Idea

The textbook example creates several threads in a loop. Each thread prints a greeting message and then exits. The main program creates all threads and then terminates.

Pthreads Example Code

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS 10

void *print_hello_world(void *tid)
{
    printf("Hello World. Greetings from thread %ld\n", (long) tid);
    pthread_exit(NULL);
}

int main(void)
{
    pthread_t threads[NUMBER_OF_THREADS];
    int status;
    long i;

    for (i = 0; i < NUMBER_OF_THREADS; i++) {
        printf("Main here. Creating thread %ld\n", i);
        status = pthread_create(&threads[i], NULL, print_hello_world, (void *) i)
            ;
        if (status != 0) {
            printf("Oops. pthread_create returned error code %d\n", status);
            exit(-1);
        }
    }

    pthread_exit(NULL);
}
```

What the Program Does

- The main thread creates 10 worker threads.
- Each new thread prints its identifier.
- After printing, each thread exits.
- The main thread also exits after creating the threads.

Nondeterministic Output

The order in which the messages appear is **not fixed**. Different runs may produce different interleavings because thread scheduling is nondeterministic.

11. Why the Example Matters

Programming Style

The example shows the standard Pthreads style:

- define a thread function,
- create threads with `pthread_create`,
- optionally wait for them using `pthread_join`,
- let each thread end using `pthread_exit`.

12. Main Conclusion

Takeaway

Pthreads gives a portable standard interface for multithreaded programming on UNIX-like systems. Its basic operations cover thread creation, termination, waiting, yielding, and attribute management.

13. Quick Review

Exam-Focused Points

- Pthreads is the IEEE POSIX thread standard.
- It is designed to make threaded programs portable across UNIX systems.
- Every thread has an identifier, registers, a program counter, and attributes.
- `pthread_create` creates a new thread.
- `pthread_exit` terminates the calling thread.
- `pthread_join` waits for a specific thread to finish.
- `pthread_yield` voluntarily gives the CPU to another thread.
- `pthread_attr_init` and `pthread_attr_destroy` manage attribute structures.
- Output order in multithreaded programs is often nondeterministic.